Part 2: Indexing and Evaluation

Group members: Júlia Hernan (u233681), Alan Le Roux (u233515), Alba Miró (u233727)

GitHub repository: https://github.com/juliahupf/RIAW.git

1. Introduction

This report describes the second phase of our search engine. Starting from the 28,080 documents pre-processed in Part 1, we build an inverted index, answer conjunctive (AND) queries and rank the results with TF-IDF. Then we implement seven evaluation metrics, apply them to the two validation queries and to five test queries of our own, for which we create the relevance judgments ourselves. The last sections analyse what the metrics say about the system and which problems it has.

The search code is in `myapp/search/algorithms.py`, the metrics in `myapp/search/evaluation.py`, and all the experiments in `project_progress/part_2/part2.ipynb`. Every output shown in this report is a capture of a cell of that notebook. Section 5 explains how to reproduce them.

2. Indexing

2.1 Query pre-processing

A query must be processed exactly like the documents: if a user writes "running" and the index stores the stem `run`, there is no match. The function `build_terms(text)` therefore repeats the pipeline of Part 1: case folding, URL removal, punctuation replaced by spaces, splitting on whitespace, NLTK English stop-word removal, removal of single letters (digits are kept) and Porter stemming. For example, "men running shoes" becomes `['men', 'run', 'shoe']`. The function is copied from the Part 1 notebook because code inside a notebook cannot be imported from a Python module.

2.2 Inverted index and conjunctive queries (Task 1)

`create_inverted_index(docs, field)` builds the index on `all_terms`, the field that Part 1 created by merging title, description, brand, category, sub_category and product_details. The statement suggests storing, for each term, the list of documents that contain it. We store that list as the keys of a dictionary, and the number of occurrences of the term in each document (term frequency) as the values, because TF-IDF needs it: `{term: {pid: tf}}`.

`conjunctive_match(query_terms, index)` implements the AND semantics required by the statement: a document is returned only if it contains every query term. It intersects the posting lists of the query terms, starting with the shortest one, because the result can never be larger than that list. If a query term is not in the vocabulary, no document can contain it and the result is empty.

```python
index, idf, doc_norms = create_tfidf_index(docs)
print("Vocabulary size:", len(index))
print("Postings of 'kurta' (first 3):", list(index["kurta"].items())[:3])

terms = build_terms("women cotton kurta")
print("Query terms:", terms)
print("Documents with each term:", {t: len(index[t]) for t in terms})
print("Documents with all terms:", len(conjunctive_match(terms, index)))
```

```
Vocabulary size: 6768
Postings of 'kurta' (first 3): [('ETHFYB46QHB6PMYG', 7), ('KTAFTMYM2VYNZACZ', 5), ('ETHFSRVEN6JGSBYG', 4)]
Query terms: ['women', 'cotton', 'kurta']
Documents with each term: {'women': 13807, 'cotton': 19918, 'kurta': 1196}
Documents with all terms: 575
```

Output 1. Building the index and an example of a conjunctive query.

Output 1 shows that the vocabulary has **6,768 terms**, and three postings of `kurta`: each pid comes with the number of times the term appears in that product. For "women cotton kurta", `women` appears in 13,807 documents, `cotton` in 19,918 and `kurta` in 1,196, but only **575 documents** contain the three terms. The AND filter discards more than half of the documents that contain the rarest term, which shows how strict conjunctive queries are.

2.3 Test queries (Task 2)

Following the hint of the statement, we chose the query words by popularity. Output 2 lists the 15 terms with the highest document frequency (df), with their collection frequency (cf, total occurrences) and their idf (Section 2.4). These terms are useless as query words: `cloth` appears in 27,722 of the 28,080 documents (98.7%), so its idf is 0.020 and it cannot separate one product from another. Most of them come from the category ("Clothing and Accessories") and from the field names of product_details (`color`, `fabric`, `pattern`, `wash`, `care`), which appear in almost every product. In other words, the most popular terms of the collection describe the structure of the data, not the products.

```
popularity = pd.DataFrame({"df": {t: len(p) for t, p in index.items()}})
popularity["cf"] = [sum(index[t].values()) for t in popularity.index]
popularity["idf"] = [round(idf[t], 3) for t in popularity.index]
popularity.sort_values("df", ascending=False).head(15)
```

	df	cf	idf
cloth	27722	31084	0.019
accessori	27189	29204	0.047
color	26862	37498	0.064
fabric	26450	57370	0.086
pattern	25783	26595	0.123
care	22642	23746	0.311
wear	20971	31691	0.421
wash	20528	26831	0.452
cotton	19918	34079	0.495
suitabl	19309	19507	0.540
western	19118	19189	0.555
sleev	18901	51332	0.571
fit	18834	39734	0.576
type	18189	38731	0.626
ideal	17198	17770	0.707

Output 2. The 15 terms with the highest document frequency.

We therefore looked at popular words that describe a product, a garment type or a target customer (Output 3). They range from very common terms, such as `men` and `women`, with an idf close to 1, to rarer and more discriminative ones, such as `run`, `sneaker` and `shoe`, with an idf above 5. Each query mixes both kinds, so that we can test whether the ranking relies on the informative terms. Note that `tshirt` appears in only 2,648 documents although T-shirts are the most common product; Section 3.6 explains why.

```
product_words = ["cotton", "men", "women", "shirt", "shoe", "run", "polo", "neck", "tshirt", "jacket", "hood",
                 "winter", "kurta", "sneaker", "white"]
popularity.loc[product_words].sort_values("df", ascending=False)
```

	df	cf	idf
cotton	19918	34079	0.495
shirt	16770	50038	0.744
neck	15190	57363	0.886
men	14250	28248	0.979
women	13807	28041	1.024
white	3937	6626	2.834
polo	3054	11489	3.201
winter	2868	3287	3.291
tshirt	2648	4009	3.407
hood	2161	3600	3.700
kurta	1196	4202	4.553
jacket	1110	3338	4.661
run	792	916	5.148
sneaker	780	803	5.170
shoe	526	1346	5.738

Output 3. Popularity of the product words used to build the test queries.

The five test queries (Output 4) were chosen to cover different situations:

- **Query 3, "women cotton kurta":** a popular ethnic product combined with a gender and a fabric.
- **Query 4, "men running shoes":** footwear, a small part of the corpus (920 products in Men's Footwear).
- **Query 5, "polo neck tshirt men":** a very popular garment, written as "tshirt" while titles write "T-Shirt".
- **Query 6, "winter hooded jacket men":** an attribute that is stored in product_details (Hooded).
- **Query 7, "white sneakers women":** a colour combined with a rare product type.

```
TEST_QUERIES = {
    3: "women cotton kurta",
    4: "men running shoes",
    5: "polo neck tshirt men",
    6: "winter hooded jacket men",
    7: "white sneakers women",
}
VALIDATION_QUERIES = {
    1: "women full sleeve sweatshirt cotton",
    2: "men slim jeans blue",
}
ALL_QUERIES = {**VALIDATION_QUERIES, **TEST_QUERIES}

results = {q: search_in_corpus(text, index, idf, doc_norms) for q, text in ALL_QUERIES.items()}
pd.DataFrame([{"query_id": q, "query": ALL_QUERIES[q], "terms": build_terms(ALL_QUERIES[q]), "retrieved": len(results[q])}
              for q in ALL_QUERIES])
```

	query_id	query	terms	retrieved
0	1	women full sleeve sweatshirt cotton	[women, full, sleev, sweatshirt, cotton]	500
1	2	men slim jeans blue	[men, slim, jean, blue]	222
2	3	women cotton kurta	[women, cotton, kurta]	575
3	4	men running shoes	[men, run, shoe]	76
4	5	polo neck tshirt men	[polo, neck, tshirt, men]	203
5	6	winter hooded jacket men	[winter, hood, jacket, men]	292
6	7	white sneakers women	[white, sneaker, women]	76

Output 4. Validation (1, 2) and test (3 to 7) queries, their terms and the number of documents retrieved.

The number of retrieved documents depends on how rare the rarest term is and on how often the terms appear together. Queries 4 and 7 retrieve only 76 documents each, because `shoe` and `sneaker` are rare, while Query 3 retrieves 575, because kurtas are numerous and almost all of them are described with `women` and `cotton`.

2.4 Ranking with TF-IDF (Task 3)

We use the vector space model: the query and every document are vectors with one weight per term, and documents are ranked by the cosine of the angle between both vectors. The weights follow the TF-IDF scheme with a logarithmic term frequency, in base 2 as in the theory sessions. The same formulas are applied to documents and queries.

Term frequency weight. A term that appears ten times does not make a document ten times more relevant, so the raw count tf(t, d) is dampened with a logarithm (tf = 1 gives 1, tf = 2 gives 2, tf = 4 gives 3):

$$\text{tf_w}(t, d) = 1 + \log_2 \text{tf}(t, d) \quad \text{if tf}(t, d) > 0, \text{and 0 otherwise} \quad (1)$$

Inverse document frequency. N is the number of documents (28,080) and df(t) the number of documents that contain t. Rare terms get a high idf and very common terms an idf close to 0:

$$\text{idf}(t) = \log_2(N / \text{df}(t)) \quad (2)$$

TF-IDF weight of a term in a document d or in the query q:

$$w(t, d) = \text{tf_w}(t, d) \cdot \text{idf}(t) \qquad w(t, q) = \text{tf_w}(t, q) \cdot \text{idf}(t) \quad (3)$$

Score. The cosine similarity divides the dot product by the length (norm) of both vectors, so that a long document does not win just because it contains many terms:

$$\text{score}(q, d) = \Sigma_{t \in q}\, w(t, q) \cdot w(t, d) \;/\; (\,|q| \cdot |d|\,), \quad |d| = \sqrt{(\,\Sigma_{t \in d}\, w(t, d)^2\,)} \qquad (4)$$

The formulas are implemented in four functions:

- `tf_weight(tf)` returns 1 + log2(tf), formula (1).
- `create_tfidf_index(docs)` builds the index and computes, once and before any query, the idf of every term (formula 2) and the norm |d| of every document (formulas 3 and 4).
- `score_tfidf(...)` weights the query terms (their tf in the query is counted with `Counter`), computes |q| and the cosine of every document that passed the AND filter (formulas 3 and 4).
- `search_in_corpus(query, ...)` runs `build_terms`, the AND filter and `score_tfidf`, and sorts by decreasing score. Ties are broken by pid, so that every run returns the same ranking.

Worked example. For "women cotton kurta" every term appears once in the query, so its weight equals its idf: `kurta` gets log2(28080 / 1196) = 4.553, `women` gets 1.024 and `cotton` 0.495. The query norm is √(4.553² + 1.024² + 0.495²) = 4.693, and `kurta` alone accounts for 94% of the squared length of the query vector. The ranking is therefore driven by the rare term, which is the intended behaviour of idf.

```
def top_results(query_id, n=10):
    rows = [{"rank": i + 1, "score": round(score, 3), "pid": pid, "title": " ".join(doc_store[pid]["title"].split()),
             "brand": doc_store[pid]["brand"], "selling_price": doc_store[pid]["selling_price"],
             "average_rating": doc_store[pid]["average_rating"]}
            for i, (pid, score) in enumerate(results[query_id][:n])]
    return pd.DataFrame(rows)

for q, text in TEST_QUERIES.items():
    print(f"Query {q}: {text}")
    display(top_results(q))
```

```
Query 3: women cotton kurta
```

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.522	KTAFAKGUDJEB5DR7	Women Solid Cotton Blend Straight Kurta (Black)	Purple Sta	719.0	2.9
1	2	0.505	ETHFPDUMHA2BTSZH	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
2	3	0.505	ETHFPDUMHFZMYHCX	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
3	4	0.505	ETHFPDUMQZ9EZJEH	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
4	5	0.505	ETHFPDUMTE4P2QW9	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
5	6	0.505	ETHFPDUMZXGVUJFT	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
6	7	0.505	ETHFPDUMTGC3VF9M	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
7	8	0.504	ETHFPDUMFNYH3EUX	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
8	9	0.501	ETHFPDUMEWUYJGKR	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
9	10	0.501	ETHFPDUMQAXJKDWZ	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9

```
Query 4: men running shoes
```

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.599	SHOEP5GUQYYFKYSN	Running Shoes For Women (White)	Vector	990.0	4.1
1	2	0.598	SHOESS9ZEZ6TEQUJ	Running Shoes For Men (Multicolor)	Vector	1440.0	4.3
2	3	0.598	SHOESS9ZGMH7CURM	Running Shoes For Men (Multicolor)	Vector	1440.0	3.7
3	4	0.438	SHOEWQKVSGFGXPQF	Running Shoes For Men (Blue)	Vector	1520.0	3.3
4	5	0.429	SHOEKNRJGEYJCDQK	SPRINT Running Shoes For Men (Blue, Grey)	Vector	490.0	3.7
5	6	0.428	SHOEKNRJ34UCMAPA	SPRINT Running Shoes For Men (Black, Grey)	Vector	490.0	3.7
6	7	0.407	SHOFWFVWRSNK5AZJ	Running Shoes For Women (Multicolor)	Bone	474.0	3.9
7	8	0.403	SHOEYF5UHUUH5ADZ	Running Shoes For Women (Blue, Orange)	Vector	990.0	4.3
8	9	0.354	SHOFTZFPNCU4VWHK	Running Shoes For Men (Black)	Bone	379.0	4.1
9	10	0.349	SHOFTZFQP8W9G86F	Running Shoes For Women (Black)	Bone	341.0	4.1

Output 5. First ten results of Queries 3 and 4.

Query 5: polo neck tshirt men

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.544	TSHFHTGUQBXMZR6W	Self Design Men Polo Neck Blue T-Shirt	ATHL	322.0	3.8
1	2	0.544	TSHFHTGUWGNGGSRC	Self Design Men Polo Neck Dark Blue T-Shirt	ATHL	322.0	3.8
2	3	0.542	TSHFGQGQZPVZHD2S	Solid Men Polo Neck Black T-Shirt	Pu	719.0	4.1
3	4	0.538	TSHFHTGBHE7HGXKT	Solid Men Polo Neck Yellow T-Shirt	ATHL	322.0	3.3
4	5	0.538	TSHFHTGBXB3ENHWH	Solid Men Polo Neck Yellow T-Shirt	ATHL	322.0	3.3
5	6	0.538	TSHFHTGKJ4R5YZ5H	Color Block Men Polo Neck Green T-Shirt	ATHL	322.0	3.3
6	7	0.537	TSHFHTGGKMW9CUBC	Color Block Men Polo Neck Blue T-Shirt	ATHL	322.0	3.3
7	8	0.533	TSHFHTGKXCZFQV6C	Color Block Men Polo Neck Green T-Shirt	ATHL	322.0	3.3
8	9	0.532	TSHFHTGGHEHSW2VX	Color Block Men Polo Neck Blue T-Shirt	ATHL	322.0	3.3
9	10	0.532	TSHFHTGKR6WZPYK2	Color Block Men Polo Neck Green T-Shirt	ATHL	322.0	3.3

Query 6: winter hooded jacket men

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.684	JCKFRP3HDZFMAPTW	Full Sleeve Printed Men Jacket	ECKO Unl	1539.0	2.5
1	2	0.683	JCKFRP3HWFS5ZGZM	Full Sleeve Solid Men Jacket	ECKO Unl	1577.0	3.3
2	3	0.682	JCKFRP3GHXH8PTFH	Full Sleeve Solid Men Jacket	ECKO Unl	1799.0	3.7
3	4	0.674	JCKFRP3HTQ977GDH	Full Sleeve Solid Men Jacket	ECKO Unl	1874.0	3.5
4	5	0.664	JCKFRP3GFRQHVDHD	Full Sleeve Solid Men Jacket	ECKO Unl	1847.0	3.4
5	6	0.663	JCKFRP3HKNWD9TRJ	Full Sleeve Solid Men Jacket	ECKO Unl	1986.0	3.8
6	7	0.630	JCKFYQGHVKD6XNCA	Full Sleeve Solid Men Casual Jacket	MASH UNLIMIT	949.0	3.2
7	8	0.620	JCKFMV5XJJQMNASZ	Sleeveless Solid Men Casual Jacket	VOXA	598.0	4.3
8	9	0.614	JCKFRP3GJRQPHNSA	Full Sleeve Solid Men Casual Jacket	ECKO Unl	1516.0	3.7
9	10	0.606	JCKFYQHZAVFQZRYN	Full Sleeve Solid Men Casual Jacket	MASH UNLIMIT	949.0	3.2

Query 7: white sneakers women

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.342	TSHFYNNX8EWBAFQF	Printed Women Round Neck White T-Shirt	Free Authori	599.0	3.9
1	2	0.307	SHTFMRH8SVHUCHH6	Women Slim Fit Solid Casual Shirt	Solid Styl	699.0	4.0
2	3	0.294	TSHEXH59JKHGZYTM	Solid Women Round Neck White, Blue T-Shirt	Fairdea	284.0	2.6
3	4	0.262	TSHFDT65DYMCGHNG	Printed Women Round Neck White T-Shirt	REEB	980.0	3.5
4	5	0.260	TSHFW7GFFHQCZZA5	Solid Women Collared Neck White T-Shirt (Pack ...	ModeWa	495.0	5.0
5	6	0.258	TSHFW7GF2JHTWHUZ	Solid Women Collared Neck White, Blue T-Shirt ...	ModeWa	495.0	3.8
6	7	0.257	TSHFW7GHBZTYWZCZ	Solid Women Collared Neck White, Black T-Shirt...	ModeWa	495.0	3.8
7	8	0.257	TSHFW7GYGCWUQMCC	Solid Women Collared Neck White, Black T-Shirt...	ModeWa	495.0	3.8
8	9	0.256	TSHFW7G3JC6HP8FZ	Solid Women Collared Neck White, Blue T-Shirt ...	ModeWa	495.0	3.8
9	10	0.251	TSHFW7G3RAZ2SSX3	Solid Women Collared Neck White, Red T-Shirt (...	ModeWa	495.0	3.8

Output 6. First ten results of Queries 5, 6 and 7.

Outputs 5 and 6 show the ranking produced for each test query. In **Query 3** the first result is a single kurta (score 0.522), followed by nine colour variants of the same "Women Kurta and Pyjama Set Pure Cotton" with almost identical scores (0.501 to 0.505): their texts are the same except for the colour, so TF-IDF cannot separate them. In **Query 4** the first result is a running shoe for women (0.599), just above two men's shoes (0.598), and three more women's shoes appear at ranks 7, 8 and 10: the query asks for men, but the ranking barely distinguishes gender. In **Query 5** all ten results are men's polo neck T-shirts, seven of them from the same brand (ATHL). In **Query 6** the scores are the highest of all queries (0.606 to 0.684), yet the first nine jackets have no hood; only the tenth does. In **Query 7** the scores are the lowest (0.251 to 0.342) and the ten results are white T-shirts and a shirt, not sneakers. Section 3 measures these observations, and Section 3.6 explains their causes.

3. Evaluation

3.1 Evaluation metrics (Task 1)

All metrics are in `myapp/search/evaluation.py` and follow the definitions of Practice Session 2. Each function receives the true labels `y_true` (1 relevant, 0 not relevant) and the scores `y_score` of the engine. The helper `rank_labels` sorts the labels by decreasing score with a stable sort, which is the first step of every metric. In the formulas, rel_i is the label at rank i, K the cut-off and R the total number of relevant documents. As a running example we use the ranked labels 0, 1, 0, 1, 1 with R = 3; the notebook checks the metrics on these cases with assert statements (Output 7).

Precision@K (`precision_at_k`) is the share of the first K results that are relevant, that is, what the user finds useful on the first page. It ignores the order inside the top K and the relevant documents outside it. Example: P@5 = 3/5 = 0.600.

$$\mathrm{P@K} = (1 \,/\, \mathrm{K}) \cdot \Sigma_{\mathrm{i}=1..\mathrm{K}}\, \mathrm{rel_i} \quad (5)$$

Recall@K (`recall_at_k`) is the share of all relevant documents that appear in the top K. The function has a parameter `total_relevant` so that R also counts the relevant documents the AND filter never returned; without it, recall would be overestimated. Its maximum value is K/R. Example: R@5 = 3/3 = 1.000.

$$\mathrm{R@K} = (1 \,/\, \mathrm{R}) \cdot \Sigma_{\mathrm{i}=1..\mathrm{K}}\, \mathrm{rel_i} \quad (6)$$

F1@K (`f1_at_k`) is the harmonic mean of both; it is only high when precision and recall are both high, and it inherits the K/R limit of recall. Example: F1@5 = 2 · 0.6 · 1 / 1.6 = 0.750.

$$\mathrm{F1@K} = 2 \cdot \mathrm{P@K} \cdot \mathrm{R@K} \,/\, (\,\mathrm{P@K} + \mathrm{R@K}\,) \quad (7)$$

Average Precision@K (`avg_precision_at_k`) adds the precision at each position where a relevant document appears, so relevant documents placed early weigh more. As in Practice Session 2, it divides by R_K, the number of relevant documents found in the top K; as a consequence, relevant documents that are never retrieved do not lower it. Example: AP@5 = (1/2 + 2/4 + 3/5) / 3 = 0.533.

$$\mathrm{AP@K} = (1 \,/\, \mathrm{R_K}) \cdot \Sigma_{\mathrm{i}=1..\mathrm{K}}\, \mathrm{P@i} \cdot \mathrm{rel_i} \quad (8)$$

Mean Average Precision (`map_at_k`) is the mean AP over the set of queries Q. It gives one number for the ranking quality of the whole system, but a very good query can hide a very bad one.

$$\mathrm{MAP@K} = (1 \,/\, |\mathrm{Q}|) \cdot \Sigma_{\mathrm{q} \in \mathrm{Q}}\, \mathrm{AP@K(q)} \quad (9)$$

Reciprocal Rank and MRR (`rr_at_k`, `mrr_at_k`) only look at the position of the first relevant document (found with np.argmax). They model a user who needs just one good answer and ignore everything after it. Example: RR = 1/2 = 0.500.

$$\mathrm{RR@K} = 1 \,/\, \mathrm{rank_{first}} \ \text{ if } \mathrm{rank_{first}} \le \mathrm{K}, \text{ and } 0 \text{ otherwise} \quad (10)$$

$$\mathrm{MRR@K} = (1 \,/\, |\mathrm{Q}|) \cdot \Sigma_{\mathrm{q} \in \mathrm{Q}}\, \mathrm{RR@K(q)} \quad (11)$$

NDCG@K (`dcg_at_k`, `ndcg_at_k`) discounts the gain of each document logarithmically with its position and divides by the DCG of the ideal ranking (IDCG), so the result is between 0 and 1. It is designed for graded relevance; we use formula 7 of the practice, with gain 2^rel − 1, which equals the label when relevance is binary, so it cannot separate a perfect match from a partial one. The parameter

`ideal_labels` builds the ideal ranking with all relevant documents, including those not retrieved. Example: DCG@5 = 1/log2(3) + 1/log2(5) + 1/log2(6) = 1.449, IDCG@5 = 1 + 1/log2(3) + 1/log2(4) = 2.131, NDCG@5 = 0.680.

$$\mathrm{DCG@K} = \Sigma_{i=1..K} \left(2^{rel_i} - 1 \right) / \log_2(i + 1) \qquad \mathrm{NDCG@K} = \mathrm{DCG@K} / \mathrm{IDCG@K} \quad (12)$$

```python
y_true = np.array([0, 1, 0, 1, 1])
y_score = np.array([0.9, 0.5, 0.6, 0.7, 0.2])   # ranked labels: 0, 1, 0, 1, 1

assert precision_at_k(y_true, y_score, 2) == 1 / 2
assert recall_at_k(y_true, y_score, 2) == 1 / 3
assert np.isclose(f1_at_k(y_true, y_score, 2), 2 * (1 / 2) * (1 / 3) / (1 / 2 + 1 / 3))
assert np.isclose(avg_precision_at_k(y_true, y_score, 5), (1 / 2 + 2 / 4 + 3 / 5) / 3)
assert rr_at_k(y_true, y_score, 5) == 1 / 2
assert rr_at_k(y_true, y_score, 1) == 0
assert ndcg_at_k(y_true, y_true, 5) == 1
assert recall_at_k(y_true, y_score, 5, total_relevant=6) == 1 / 2   # relevant documents not retrieved count
print("Metric checks passed")
```

```
Metric checks passed
```

Output 7. Checks of the metrics on cases computed by hand.

How the rankings are evaluated. The notebook function `evaluate_query` computes all metrics of one query for K = 5, 10 and 20, and `evaluate` adds MAP and MRR over a set of queries. For the validation queries only the 40 documents of `validation_labels.csv` are judged, so only those documents enter the ranking, in the order given by our engine. For our test queries every document of the corpus has a label (Section 3.3), so the whole result list is evaluated. In both cases R counts the relevant documents that the AND filter did not return.

3.2 Results on the validation queries (Task 2)

```
pd.options.display.float_format = "{:.3f}".format   # the statement asks for three decimals
validation = pd.read_csv(VALIDATION_FILE)
validation_labels = {q: dict(zip(group["pid"], group["labels"])) for q, group in validation.groupby("query_id")}
validation_table, validation_summary = evaluate(VALIDATION_QUERIES, validation_labels)
display(validation_table.round(3).T)
display(validation_summary.round(3))
```

query_id	1	2
P@5	1.000	0.800
R@5	0.385	0.400
F1@5	0.556	0.533
AP@5	1.000	1.000
RR@5	1.000	1.000
NDCG@5	1.000	0.869
P@10	1.000	0.600
R@10	0.769	0.600
F1@10	0.870	0.600
AP@10	1.000	0.948
RR@10	1.000	1.000
NDCG@10	1.000	0.716
P@20	0.650	0.300
R@20	1.000	0.600
F1@20	0.788	0.400
AP@20	0.980	0.948
RR@20	1.000	1.000
NDCG@20	0.995	0.716

k	MAP	MRR
5	1.000	1.000
10	0.974	1.000
20	0.964	1.000

Output 8. Metrics of the validation queries.

3.3 Ground truth for the test queries (Task 3a)

To compute recall correctly we need every relevant document of the corpus, also those that the engine misses, and judging 28,080 documents by hand for five queries is not feasible. We therefore wrote down each information need as an explicit rule and applied it to the whole collection (`RELEVANCE_RULES` in Output 9). A document is relevant if:

- **Query 3:** the title contains "women" and "kurta", and the Fabric detail contains cotton (a kurta alone or in a set).
- **Query 4:** the title contains "running shoes" and "for men".
- **Query 5:** the title describes a men's polo neck T-shirt, however "T-shirt" is written in the query.
- **Query 6:** it is a men's jacket of the sub_category Winter Wear and its Hooded detail is Yes.
- **Query 7:** the title contains "sneakers for women" and the Color detail contains white.

The rules use the title and the structured product_details, which describe the product itself. The description is not used, because it often mentions other products ("pair it with sneakers"). We then read

the top 20 results of each query to check that the rule agreed with our own judgment. The labels are saved in `test_labels.csv`, with the same format as `validation_labels.csv` (title, pid, query_id, labels): every relevant document plus the non-relevant documents of each top 20, 1,769 rows in total.

```python
def details(doc):
    flat = {}
    for item in doc["product_details"] or []:
        flat.update(item)
    return flat

def has(pattern, text):
    return re.search(pattern, text or "", re.IGNORECASE) is not None

def clean_title(doc):
    return " ".join(doc["title"].split())

RELEVANCE_RULES = {
    # a women's kurta (alone or in a set) whose fabric is cotton
    3: lambda d: has(r"\bwomen\b", d["title"]) and has(r"\bkurta\b", d["title"]) and has("cotton", details(d).get("Fabric")),
    # running shoes made for men, not for women
    4: lambda d: has(r"running shoes", d["title"]) and has(r"\bfor men\b", d["title"]),
    # a men's T-shirt with polo neck, however the word "T-shirt" is written
    5: lambda d: has(r"\bmen\b.*polo neck.*t-shirt", clean_title(d)),
    # a men's jacket for winter that really has a hood
    6: lambda d: has(r"\bmen\b", d["title"]) and has(r"\bjacket\b", d["title"]) and d["sub_category"] == "Winter Wear"
                 and details(d).get("Hooded") == "Yes",
    # women's sneakers in white
    7: lambda d: has(r"sneakers for women", clean_title(d)) and has("white", details(d).get("Color")),
}

judged_rows = []
for q in TEST_QUERIES:
    relevant = {d["pid"] for d in docs if RELEVANCE_RULES[q](d)}
    top_20 = {pid for pid, _ in results[q][:20]}
    judged_rows += [{"title": clean_title(doc_store[pid]), "pid": pid, "query_id": q, "labels": int(pid in relevant)}
                    for pid in sorted(relevant | top_20)]

test_labels = pd.DataFrame(judged_rows)
test_labels.to_csv("test_labels.csv", index=False)   # same format as validation_labels.csv

test_labels_by_query = {}
for q in TEST_QUERIES:
    relevant = set(test_labels[(test_labels["query_id"] == q) & (test_labels["labels"] == 1)]["pid"])
    test_labels_by_query[q] = {pid: int(pid in relevant) for pid, _ in results[q]} | {pid: 1 for pid in relevant}

pd.DataFrame([{"query_id": q, "query": TEST_QUERIES[q], "retrieved": len(results[q]),
               "relevant in corpus": sum(labels.values()),
               "relevant retrieved": sum(labels[pid] for pid, _ in results[q]),
               "relevant in top 20": sum(labels[pid] for pid, _ in results[q][:20])}
              for q, labels in test_labels_by_query.items()])
```

	query_id	query	retrieved	relevant in corpus	relevant retrieved	relevant in top 20
0	3	women cotton kurta	575	322	322	20
1	4	men running shoes	76	23	23	10
2	5	polo neck tshirt men	203	1326	186	20
3	6	winter hooded jacket men	292	49	49	4
4	7	white sneakers women	76	3	3	0

Output 9. Relevance rules and number of relevant documents per test query.

The table at the bottom of Output 9 already shows two very different problems. For Queries 3, 4, 6 and 7, "relevant retrieved" equals "relevant in corpus": the AND filter returns **all** the relevant documents, so any error is a ranking error. Their share in the result list, however, varies a lot: 322 of 575 (56%) for Query 3, 23 of 76 (30%) for Query 4, 49 of 292 (17%) for Query 6 and only 3 of 76 (4%) for Query 7. When we inspected the 253 non-relevant results of Query 3, 242 were dhotis and ethnic dresses, which

contain `kurta` only because their sub_category is "Kurtas, Ethnic Sets and Bottoms". Query 5 is the opposite case: almost everything it retrieves is relevant (186 of 203, 92%), but 1,140 of its 1,326 relevant T-shirts are never retrieved, a recall of only 14% over the whole list. The number of relevant documents also ranges from 3 to 1,326, which matters when reading the recall-based metrics below.

3.4 Results on the test queries

```
test_table, test_summary = evaluate(TEST_QUERIES, test_labels_by_query)
display(test_table.round(3).T)
display(test_summary.round(3))
```

query_id	3	4	5	6	7
P@5	1.000	0.800	1.000	0.000	0.000
R@5	0.016	0.174	0.004	0.000	0.000
F1@5	0.031	0.286	0.008	0.000	0.000
AP@5	1.000	0.679	1.000	0.000	0.000
RR@5	1.000	0.500	1.000	0.000	0.000
NDCG@5	1.000	0.661	1.000	0.000	0.000
P@10	1.000	0.600	1.000	0.100	0.000
R@10	0.031	0.261	0.008	0.020	0.000
F1@10	0.060	0.364	0.015	0.034	0.000
AP@10	1.000	0.703	1.000	0.100	0.000
RR@10	1.000	0.500	1.000	0.100	0.000
NDCG@10	1.000	0.573	1.000	0.064	0.000
P@20	1.000	0.500	1.000	0.200	0.000
R@20	0.062	0.435	0.015	0.082	0.000
F1@20	0.117	0.465	0.030	0.116	0.000
AP@20	1.000	0.642	1.000	0.167	0.000
RR@20	1.000	0.500	1.000	0.100	0.000
NDCG@20	1.000	0.512	1.000	0.147	0.000

k	MAP	MRR
5	0.536	0.500
10	0.561	0.520
20	0.562	0.520

Output 10. Metrics of the five test queries, and MAP and MRR over the five queries.

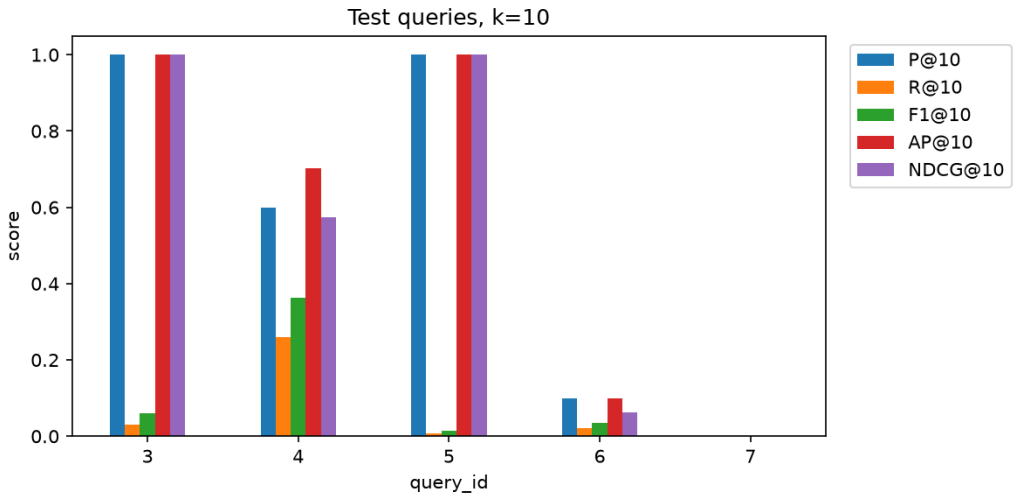


Figure 1. P, R, F1, AP and NDCG at K = 10 for each test query.

Figure 1 shows three very different behaviours. **Queries 3 and 5** have P@10, AP@10 and NDCG@10 equal to 1.000: the first ten results are all relevant. Their R@10 (0.031 and 0.008) and F1@10 (0.060 and 0.015) are almost zero, but this is not a failure of the ranking: with 322 and 1,326 relevant documents, ten results can find at most 10/322 = 0.031 and 10/1326 = 0.008 of them, which are exactly the values obtained. Recall at a small K is uninformative when a query has hundreds of relevant documents.

Query 4 is the only one where all five bars are of medium height. P@10 = 0.600 means that 6 of the first 10 results are relevant. AP@10 (0.703) is higher than P@10 because the relevant documents are concentrated at ranks 2 to 6 (Output 5). NDCG@10 (0.573) is lower than AP@10: NDCG compares with an ideal ranking of ten relevant documents and is penalised by the non-relevant documents at ranks 1, 7, 8 and 10, especially the one at rank 1, whose discount is the smallest. RR@10 = 0.500 because the first relevant document is at rank 2. R@10 = 0.261 is the highest recall at K = 10, since there are only 23 relevant documents.

Query 6 has values close to zero (P@10 = 0.100, AP@10 = 0.100, NDCG@10 = 0.064, RR@10 = 0.100): only one of the first ten results is relevant and it is at rank 10. **Query 7** has no bars at all: no relevant document is in the top 10. The per-query view is essential here, because MAP@10 = 0.561 and MRR@10 = 0.520 (Output 10) describe none of the five queries.

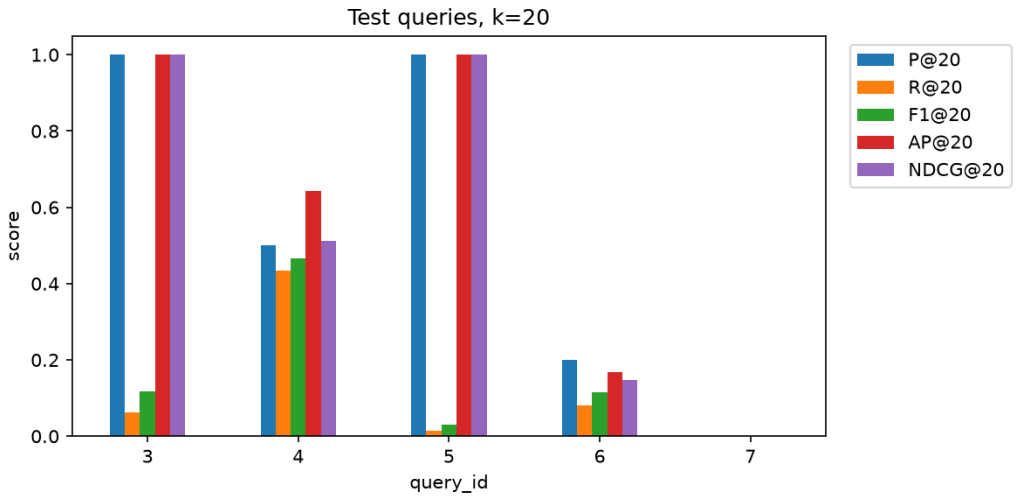


Figure 2. The same metrics at K = 20.

Figure 2 shows how the metrics change when the user looks at twice as many results. For Queries 3 and 5 precision stays at 1.000 for 20 results, and recall doubles (0.062 and 0.015), still limited by K/R. For Query 4 precision falls from 0.600 to 0.500 (only 4 of results 11 to 20 are relevant), while recall rises from 0.261 to 0.435, so F1 increases from 0.364 to 0.465, the best F1 of the five queries. AP@20 (0.642) and NDCG@20 (0.512) decrease because the new relevant documents arrive late. Query 6 improves slightly (P@20 = 0.200, AP@20 = 0.167) because three more relevant jackets appear, at ranks 13, 14 and 20. Query 7 remains at 0: its first relevant document is at rank 22, just outside the cut-off (Output 11). The difference between RR and AP is also visible: RR is 0.500 for Query 4 because only rank 1 is wrong, and 0.100 for Query 6 because the first relevant document is the tenth, while AP also rewards or penalises every later position.

3.5 Comments on the evaluation metrics (Task 3b)

The metrics answer different questions about the same ranking:

- **P@K** tells how useful the first page is, but not in which order the results come or how many relevant documents are left outside it. It divides by K even when fewer than K documents are retrieved.
- **R@K** tells how much of the relevant material has been found, but its maximum is K/R, so it is low whenever a query has many relevant documents (Queries 3 and 5).
- **F1@K** is only high when precision and recall are both high, so it rewards balanced rankings (Query 4 at K = 20).
- **AP@K** measures whether the relevant documents of one query are at the top. With the practice definition it cannot see relevant documents that were never retrieved.
- **MAP** summarises the system in one number, at the cost of hiding the differences between queries.
- **RR and MRR** only look at the first relevant document, which suits a user who needs one good answer.
- **NDCG@K** compares with the ideal ranking and penalises errors at the top more than errors at the bottom. Its ideal ranking includes all relevant documents, so it does notice the documents that the system misses.

Validation queries. Query 1 obtains P@10 = 1.000 and R@20 = 1.000: the first ten of the judged documents are relevant and the 13 relevant documents are almost entirely ahead of the 7 non-relevant

ones (AP@20 = 0.980). This looks excellent, but it only means that the judged documents are well ordered among themselves. In the full ranking of our engine they appear between ranks 149 and 500 of 500 (Output 11), so the first page shows documents that nobody judged.

Query 2 shows why several metrics are needed. AP@10 = 0.948 and RR = 1.000 suggest an almost perfect system, while R@20 = 0.600 and NDCG@20 = 0.716 show that 4 of the 10 relevant documents were never retrieved. Only 8 of the 20 judged documents pass the AND filter, which also explains why P@20 drops to 0.300. AP cannot see the missing documents with the definition of the practice, while NDCG can, because its ideal ranking contains all of them. Three of the four missing documents are titled "Women" (for example "Slim Women Blue Jeans") and lack the term `men`; the fourth lacks `slim`.

Test queries. MAP@10 = 0.561 and MRR@10 = 0.520 are averages of very different behaviours: two queries with perfect precision (3 and 5), one medium query (4) and two failures (6 and 7). MAP also hides the recall problem of Query 5, since its AP is 1.000. Comparing with the validation set, which has MAP@10 = 0.974, would be misleading: the validation metrics only rank 40 hand-picked documents, while the test metrics rank the full result lists.

3.6 Problems of the search system and proposed solutions (Task 3c)

Output 11 collects the measurements that support each problem below.

```
Q4 retrieved documents for women: (39, 76)
Q4 top 20 for women: (9, 20)
Q4 retrieved that are not shoes: (23, 76)
Q6 retrieved with Hooded = No: (181, 292)
Q6 retrieved with Hooded = Yes: (68, 292)
Q6 top 20 with Hooded = No: (16, 20)
Q5 relevant, not retrieved: 1140 of 1326
Documents with T-Shirt in the title: 12247 | with term 'tshirt': 2648
Terms of 't-shirt': ['shirt'] | 'tshirt': ['tshirt'] | 'hoodie': ['hoodi'] | 'hooded': ['hood']

Different (title, brand) in the top 20:
  3 2
  4 18
  5 13
  6 9
  7 15

Empty description, corpus: 0.397
Empty description, top 20 of the test queries: 0.42
Out of stock in the top 20 of the test queries: 8
Distinct scores in the top 20: {3: 7, 4: 19, 5: 18, 6: 19, 7: 17}

Validation documents:
  Q1: judged 20, retrieved 20, relevant not retrieved 0, engine ranks 149 to 500 of 500
  Q2: judged 20, retrieved 8, relevant not retrieved 4, engine ranks 5 to 175 of 222
                      title             pid
       Slim Women Blue Jeans JEAF8CHSVCP5GUH9
Super Skinny Women Blue Jeans JEAFUZXRDYSVFK4Y
   Tapered Fit Men Blue Jeans JEAFUPSWVCTKXMHF
Super Skinny Women Blue Jeans JEAFUZXSMRFFNGC2

Category terms of the first Q4 result: ['footwear', 'men', 'footwear'] | Running Shoes For Women (White)
Q7 ranks of the relevant documents: [22, 52, 58]
Q7 top 20 with 'sneaker' in the title: (0, 20)
Q7 retrieved with 'sneaker' in the title: (3, 76)
JEAF8CHSVCP5GUH9 | url: gas-slim-men-blue-jeans | Ideal For: Women
JEAFUZXRDYSVFK4Y | url: ecko-unltd-super-skinny-men-blue-jeans | Ideal For: Women
JEAFUPSWVCTKXMHF | url: ecko-unltd-tapered-fit-men-blue-jeans | Ideal For: Men
JEAFUZXSMRFFNGC2 | url: ecko-unltd-super-skinny-men-blue-jeans | Ideal For: Women
Title says women, url says men: 12764
Title says men, url says women: 25
run idf 5.148
shoe idf 5.738
sneaker idf 5.17
white idf 2.834
women idf 1.024
```

Output 11. Measurements used in the analysis of the search system.

1. All fields have the same weight (ranking quality, field types). A term in the title counts like a term in a long description. In Query 7, white T-shirts whose description mentions sneakers are ranked above the three real white sneakers: only 3 of the 76 retrieved documents have "sneaker" in the title, none of them in the top 20, and they appear at ranks 22, 52 and 58.

> *Proposed solution:* Index the fields separately, as proposed in Part 1, and use a field-weighted score such as BM25F, giving more weight to the title and the product type.

2. Field names and values are indexed as plain words (field types, indexing). product_details is flattened into text such as "Hooded No", so the term hood also matches jackets without a hood. In

Query 6, 181 of the 292 retrieved documents have Hooded = No, against 68 with Hooded = Yes, and 16 of the top 20 have no hood.

Proposed solution: Keep attributes such as Hooded, Color or Fabric as structured fields used as filters, and map query words to them ("hooded" becomes Hooded = Yes).

3. The category text adds misleading terms (retrieval accuracy). The sub_category "Men's Footwear" also contains women's shoes, so all of them receive the term `men`: the first result of Query 4 has the category terms ['footwear', 'men', 'footwear'] although it is a shoe for women, and 39 of the 76 retrieved documents and 9 of the top 20 are for women. In the same way, "Kurtas, Ethnic Sets and Bottoms" adds `kurta` to 242 dhotis and ethnic dresses in Query 3.

Proposed solution: Index category terms in a separate low-weight field, detect gender in the query and filter by the Ideal For detail.

4. The pre-processing breaks "T-Shirt" (pre-processing, query formulation). "t-shirt" becomes `shirt`, because the single letter is dropped, while "tshirt" stays as a different term. 12,247 documents have "T-Shirt" in the title, but only 2,648 contain `tshirt`. In Query 5, 1,140 of the 1,326 relevant documents are lost, and the query "t-shirt" would also match ordinary shirts. "hooded" and "hoodie" also become different terms (`hood` and `hoodi`).

Proposed solution: Normalise compound words before tokenizing ("t-shirt" and "tshirt" to one term) and add a synonym dictionary for the fashion domain, applied at indexing or query time.

5. The AND filter is too strict (retrieval accuracy, query formulation). A single missing word removes a document. In validation Query 2, 4 of the 10 relevant documents are lost this way.

Proposed solution: Keep AND as a first pass and fall back to a softer match when there are few results, for example requiring a minimum share of the query terms, or rank with OR so that documents matching more terms score higher.

6. Titles and URLs disagree on gender (data quality). In 12,764 documents the title says "Women" while the product URL says "men". For example, JEAF8CHSVCP5GUH9 is titled "Slim Women Blue Jeans" but its URL is gas-slim-men-blue-jeans. That document is labelled relevant for "men slim jeans blue" but is not retrieved. Our own labels rely on the title and Ideal For, so they inherit this noise.

Proposed solution: Compare title, Ideal For and URL during pre-processing and flag contradictions before indexing.

7. Near-duplicates fill the first page (ranking quality). In Query 3 the top 20 contain only 2 different products (same title and brand), and only 7 different scores. P@20 = 1.000 hides that the user sees the same product many times (Output 5).

Proposed solution: Group colour variants of the same product, or diversify the results, for example with Maximal Marginal Relevance.

8. The score only uses the text (ranking quality). Rating, price and stock are not part of the score: 8 of the 100 top-20 results of the test queries are out of stock.

Proposed solution: Remove or demote out-of-stock products and combine the text score with the average rating, or learn the combination with Learning to Rank.

9. The evaluation has its own limits. The validation set only judges 40 documents, far from the first results of the engine, and documents with identical titles get different labels (for example "Slim Men

Blue Jeans" appears with label 1 and with label 0). Our labels come from explicit rules, on five queries, and are binary.

Proposed solution: Judge the top results of the engine for every new version, use more queries and graded relevance, so that NDCG can use its full scale.

4. Conclusions and implications for the next phases

- **The AND filter and the ranking fail in different queries.** For four of the five test queries the filter keeps all relevant documents and the problem is the order. For Query 5 the order is perfect but 86% of the relevant documents are never retrieved. A single metric such as MAP does not show this; precision, recall and the full result list must be read together.
- **Most errors come from how fields are turned into text.** Category names (`men`, `kurta`) and attribute names (`hood`) produce matches that do not describe the product. Field-weighted ranking and structured attribute filters are the main improvements for Part 3.
- **Pre-processing must handle domain spellings**, such as "t-shirt" and "tshirt", before any ranking model can help.
- **Non-textual signals** (stock, rating) and result diversification should enter the final score.

5. Reproducibility

The README in `project_progress/part_2` gives step-by-step instructions. The notebook `part2.ipynb` reads `processed_docs.json` from Part 1, builds the index, runs every query and writes `test_labels.csv` and the figures in `evaluation_output/`. Rankings are deterministic, because ties are broken by pid.

6. AI usage statement

For this part the lecturer allowed the use of AI. We used Claude (Anthropic) to implement the indexing, ranking and evaluation code, to propose the test queries and the relevance rules, to run the experiments and to draft this report. We reviewed the code, the relevance labels and the results, and we revised the text of the report.